KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 6 Quiz

Kalıtım ve Kompozisyon

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (10 puan)

4 • Değerlendirme Hedef 1

`kitaplik::Materyal` taban sınıfı `baslik_`, `yazar_`, `yil_` alanlarını `protected` bildirir; türetilmiş `BasiliKitap`'ın `yazdir()`'i bu alanlara doğrudan erişir. `protected` erişim belirleyicisi için hangisi DOĞRUDUR?

(a) `protected` üyeler `public` gibidir; sınıf dışındaki herhangi bir kod da (ör. `main`) bu alanlara doğrudan erişebilir.

(b) `protected` üyeler sınıfın kendisine ve ondan türeyen sınıflara açık, sınıf dışındaki koda kapalıdır; bu yüzden `BasiliKitap::yazdir()` `baslik_`'e doğrudan erişebilirken sınıf dışı kod erişemez.

(c) `protected` üyeler yalnız sınıfın kendisine açıktır; türetilmiş sınıflar bile erişemez — davranışı `private` ile birebir aynıdır.

(d) `protected` yalnızca kuruculardа geçerlidir; sıradan üye fonksiyonlarda hiçbir etkisi yoktur.

Soru 2 (15 puan)

4 • Değerlendirme Hedef 1

`BasiliKitap`, `public` kalıtımla `Materyal`'den türer ve tabanın `baslik()` erişimcisini devralır. Taban kurucusu `Materyal kuruldu`, türetilmiş kurucusu `BasiliKitap kuruldu` izleme satırı basar. Aşağıdaki kesiti inceleyiniz:

```
10    BasiliKitap bk("Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320);
11    std::cout << "Devralinan baslik(): " << bk.baslik() << '\n';
12    bk.yazdir();
```

Bu kesitin konsola bastığı çıktıyı satır satır yazınız.

Soru 3 (15 puan)

4 • Değerlendirme Hedef 2

Her kurucu ve yıkıcı bir izleme satırı basar (`Materyal kuruldu`, `BasiliKitap kuruldu`, `BasiliKitap yikildi`, `Materyal yikildi`). Aşağıdaki kesitte bir `BasiliKitap` nesnesi iç içe bir kapsam bloğunda oluşturulup blok bitince yok edilir:

```
18    std::cout << "-- kapsam basliyor --\n";
19    {
20       BasiliKitap gecici("Temiz Kod", Yazar("Robert C. Martin", "ABD"), 2008,
21                          464);
```

Soru 3 (devamı)

```
22    }
23    std::cout << "-- kapsam bitti --\n";
```

Konsola basılan çıktıyı satır satır yazınız.

Soru 4 (10 puan)

4 • Değerlendirme Hedef 2

`bk` bir `BasiliKitap` iken `Materyal m = bk;` yazılır (`m` bir `Materyal` değişkenidir, işaretçi/referans değil). Bu satır ve sonrasındaki `m.yazdir()` çağrısı için hangisi DOĞRUDUR?

(a) `m`, `bk`'nin tam bir kopyasıdır; `m.yazdir()` `BasiliKitap` sürümünü çağırır ve sayfa sayısını da basar.

(b) `m`, `bk`'nin yalnız `Materyal` alt-nesnesini kopyalar; `sayfaSayisi_` ve türetilmişe özgü davranış kesilir (nesne dilimlemesi), bu yüzden `m.yazdir()` taban sürümünü çağırır ve çıktıda sayfa sayısı bulunmaz.

(c) Bu satır derlenmez, çünkü bir türetilmiş nesne bir taban türü değişkenine değerle atanamaz.

(d) `m`, `bk`'ye bir referans olur; ikisi aynı nesneyi paylaşır ve `m.yazdir()` `BasiliKitap` sürümünü çağırır.

Soru 5 (10 puan)

4 • Değerlendirme Hedef 3

Bir `Materyal* p = new BasiliKitap(...);` işaretçisi üzerinden `delete p;` çağrılır. `~Materyal()` yıkıcısının `virtual` olup olmamasıyla ilgili hangisi DOĞRUDUR?

(a) `~Materyal()` `virtual` olmasa bile `delete p;` her iki yıkıcıyı da doğru sırada çağırır; sanal yıkıcı yalnızca bir üslup tercihidir.

(b) `~Materyal()` `virtual` DEĞİLSE `delete p;` yalnız taban yıkıcısını çağırır; türetilmişin yıkıcısı çalışmaz (tanımsız davranış, türetilmişin kaynağı sızabilir). Bu yüzden taban işaretçiyle silinebilen polimorfik bir tabanın yıkıcısı `virtual` olmalıdır.

(c) `Materyal*` üzerinden `delete` her zaman derleme hatası verir; bir türetilmiş nesne taban işaretçiyle silinemez.

(d) `virtual` yıkıcı yalnız türetilmişin yıkıcısını çağırır; tabanın yıkıcısı hiç çalışmaz, bu yüzden kaynak yine sızar.

Soru 6 (15 puan)

4 • Değerlendirme Hedef 3

`Materyal` listelerini tek döngüde gerçek türlerine göre yazdırabilmek için `yazdir()`'in çalışma-zamanı çokbiçimliliğine katılması ve `BasiliKitap`'ta güvenli biçimde geçersiz kılınması isteniyor. Aşağıdaki iskeleti, doğru anahtar sözcüklerle tamamlayacak bildirimleri yazınız:

```
class Materyal {
public:
   // (a) yazdir()'i cokbicimli yapan bildirim + (c) sanal yikici
   // ...
};
```

Soru 6 (devamı)

```
6  class BasiliKitap : public Materyal {
7  public:
8     // (b) yazdir()'i guvenli gecersiz kilan bildirim
9     // ...
10 };
```

(a) `Materyal` içinde `yazdir()`'i çokbiçimli yapan bildirimi, (b) `BasiliKitap` içinde onu güvenli geçersiz kılan bildirimi ve (c) taban işaretçiyle güvenli silme için `Materyal`'e eklenecek yıkıcı bildirimini yazınız. Her birinde hangi anahtar sözcüğün neden gerektiğini kısaca belirtiniz.

Soru 7 (10 puan)

4 • Değerlendirme Hedef 3

`Sekil` sınıfı `virtual double alan() const = 0;` bildirir; ondan `Daire` ve `Kare` türer. (a) `= 0` eki ne anlama gelir ve `Sekil`'i nasıl bir sınıf yapar? (b) `Sekil s;` neden derlenmez ama `Daire d(3.0);` derlenir? (c) `Sekil* s = &d; s->alan();` çağrısı hangi `alan()`'ı çalıştırır? Kendi cümlelerinizle açıklayınız.

Soru 8 (10 puan)

4 • Değerlendirme Hedef 4

Bir programcı `Kutuphane`'yi tasarlarken `class Kutuphane : public std::vector<std::unique_ptr<Materyal>>` biçiminde `std::vector`'dan *türetmeyi* önerir. Bu tasarım kararı için hangisi DOĞRUDUR?

(a) Doğru tasarımdır; `Kutuphane` bir materyal koleksiyonu olduğu için `std::vector`'dan türetmek en temiz yoldur.

(b) Yanlıştır; `Kutuphane` bir `std::vector` DEĞİLDİR (is-a testi başarısız), materyallere SAHİPTİR (has-a). Doğru seçim kompozisyondur: `Kutuphane` bir `std::vector<...>` üyesi (`raf_`) içerir.

(c) Fark yoktur; kalıtım ile kompozisyon aynı sonucu verdiğinden hangisi seçilirse seçilsin tasarım eşdeğerdir.

(d) Kompozisyon yalnız `Materyal` hiyerarşisi için geçerlidir; koleksiyon tutan her sınıf mutlaka bir kaptan türetilmelidir.

Soru 9 (10 puan)

4 • Değerlendirme Hedef 4

`Kutuphane`, materyalleri değer kabı `std::vector<Materyal>` yerine işaretçi kabı `std::vector<std::unique_ptr<Materyal>>` ile tutar. `hepsiniYazdir()` ise tek döngüde her `m->yazdir()`'i çağırır. Değer kabı kullanılsaydı ne yanlış giderdi? *En az iki ayrı gerekçe* vererek açıklayınız.